

EXPERIMENT 1

Student Name: ARYAN DAHIYA

Branch: AIT-CSE-AIML

Semester: 5

Subject Name: ADBMS

UID: 24BAI70697

Section/Group: 24AIT_KRG1_B

Subject Code: 24CSP-310

QUESTION A):

Write an SQL solution to calculate the total number of bank accounts belonging to each salary category. The classification categories are:

1. "Low Salary": All monthly salaries strictly less than \$20,000.
2. "Average Salary": All monthly salaries in the inclusive range [\$20,000, \$50,000].
3. "High Salary": All monthly salaries strictly greater than \$50,000. The final result table must report all three categories, even if a category contains zero accounts.

SOLUTION:

QUERY:

```
Experiment 1/postgres@PostgreSQL 14
Query Query History
30
31 CREATE TABLE Accounts (
32     account_id INT PRIMARY KEY,
33     income INT NOT NULL
34 );
35
36 INSERT INTO Accounts (account_id, income) VALUES
37 (3, 108939),
38 (2, 12747),
39 (8, 87709),
40 (6, 91738),
41 (7, 45169);
42
43 SELECT 'LOW CATEGORY' AS CATEGORY,
44        COUNT(ACCOUNT_ID) AS ACCOUNT_COUNT
45 FROM ACCOUNTS
46 WHERE INCOME <20000
47
48 UNION ALL
49
50 SELECT 'AVERAGE CATEGORY' AS CATEGORY,
51        COUNT(ACCOUNT_ID) AS ACCOUNT_COUNT
52 FROM ACCOUNTS
53 WHERE INCOME >20000 AND INCOME <=50000
54
55 UNION ALL
56
57 SELECT 'HIGH CATEGORY' AS CATEGORY,
58        COUNT(ACCOUNT_ID) AS ACCOUNT_COUNT
59 FROM ACCOUNTS
60 WHERE INCOME >50000;
61
62
```

EXPERIMENT 1

OUTPUT:

Data Output	Messages	Notifications
≡ + 📄 ▼ 📋 ▼ 🗑️ 🗄️ ⬇️ 📈 SQL		
	category text	account_count bigint
1	LOW CATEORY	1
2	AVERAGE CATEO...	1
3	HIGH CATEORY	3

QUESTION (B)

Design an Employee Management System database architecture by implementing the following operations sequentially:

1. Create and populate structural tracking tables for Departments, Employees, and Salaries.
2. Query the system using multi-table Joins to pull clear department breakdowns of employees, department and salaries, apply a 10% salary enhancement across all members of the HR department, and delete any individual salary record dropping strictly below \$30,000.
3. List all remaining active employees whose personal salary exceeds the overall company-wide average salary.

SOLUTION:

QUERY:

EXPERIMENT 1

```

Experiment 1/postgres@PostgreSQL 14
Query History
64 CREATE TABLE Departments (
65     dept_id INT PRIMARY KEY,
66     dept_name VARCHAR(50) NOT NULL
67 );
68
69 CREATE TABLE Employees (
70     emp_id INT PRIMARY KEY,
71     name VARCHAR(50) NOT NULL,
72     dept_id INT REFERENCES Departments(dept_id)
73 );
74
75 CREATE TABLE Salaries (
76     emp_id INT PRIMARY KEY REFERENCES Employees(emp_id) ON DELETE CASCADE,
77     salary INT NOT NULL
78 );
79
80
81 INSERT INTO Departments (dept_id, dept_name) VALUES
82 (10, 'HR'), (20, 'IT'), (30, 'Sales');
83
84 INSERT INTO Employees (emp_id, name, dept_id) VALUES
85 (1, 'Alice', 10), (2, 'Bob', 10), (3, 'Charlie', 20), (4, 'David', 20), (5, 'Emma', 30);
86
87 INSERT INTO Salaries (emp_id, salary) VALUES
88 (1, 50000), (2, 25000), (3, 80000), (4, 40000), (5, 45000);
89
90
91 SELECT E.*, D.*, S.*
92 FROM
93 EMPLOYEES AS E
94 JOIN
95 DEPARTMENTS AS D
96 ON
97 E.DEPT_ID = D.DEPT_ID
98 JOIN
99 SALARIES AS S
100 ON
101 E.EMP_ID = S.EMP_ID;
102
103 UPDATE SALARIES
104 SET SALARY = SALARY + SALARY * 0.10
105 WHERE EMP_ID
106 IN
107 (
108     SELECT E.EMP_ID
109     FROM EMPLOYEES AS E
110     JOIN
111     DEPARTMENTS AS D
112     ON
113     E.DEPT_ID = D.DEPT_ID
114     WHERE D.DEPT_NAME = 'HR'
115 )
116
117
118 SELECT E.NAME, D.DEPT_NAME, S.SALARY
119 FROM
120 EMPLOYEES AS E
121 JOIN
122 DEPARTMENTS AS D
123 ON
124 E.DEPT_ID = D.DEPT_ID
125 JOIN
126 SALARIES AS S
127 ON
128 E.EMP_ID = S.EMP_ID;

```

EXPERIMENT 1

OUTPUT:

Data Output

Messages

Notifications

≡+

📄

▼

📋

▼

🗑️

🗄️

⬇️

📈

SQL

	name character varying (50) 🔒	dept_name character varying (50) 🔒	salary integer 🔒
1	Charlie	IT	80000
2	David	IT	40000
3	Emma	Sales	45000
4	Alice	HR	55000
5	Bob	HR	27500

Total rows: 5

Query complete 00:00:00.068

QUESTION (C):

A pizza restaurant stores all the available pizza toppings and their ingredient costs in a table called `pizza_toppings`.

Every customer must choose exactly 3 different toppings to prepare a pizza.

Write a PostgreSQL query to generate every possible 3-topping pizza combination along with its total ingredient cost.

Requirements

Every pizza must contain exactly 3 different toppings.

The same topping cannot appear more than once in a pizza.

Different arrangements of the same toppings should be considered the same pizza.

Example:

Chicken, Pepperoni, Sausage

Pepperoni, Chicken, Sausage

Sausage, Chicken, Pepperoni

These are all the same pizza, so only one should appear.

Display the topping names separated by commas.

Display the total ingredient cost of each pizza.

Sort the output:

First by highest total cost

If two pizzas have the same cost, sort them alphabetically.

EXPERIMENT 1

SOLUTION:

QUERY:

```

1 CREATE TABLE pizza_toppings (
2     topping_name VARCHAR(50) PRIMARY KEY,
3     ingredient_cost NUMERIC(4,2) NOT NULL
4 );
5
6
7 INSERT INTO pizza_toppings (topping_name, ingredient_cost) VALUES
8 ('Pepperoni', 0.50),
9 ('Sausage', 0.70),
10 ('Chicken', 0.55),
11 ('Onions', 0.25),
12 ('Extra Cheese', 0.40);
13
14
15 SELECT
16     CONCAT(p1.topping_name, ',', p2.topping_name, ',', p3.topping_name) AS pizza,
17     (p1.ingredient_cost + p2.ingredient_cost + p3.ingredient_cost) AS total_cost
18 FROM pizza_toppings AS p1
19 JOIN pizza_toppings AS p2
20     ON p1.topping_name < p2.topping_name
21 JOIN pizza_toppings AS p3
22     ON p2.topping_name < p3.topping_name
23 ORDER BY
24     total_cost DESC
25
26

```

OUTPUT:

Data Output Messages Notifications		
SQL		
	pizza text	total_cost numeric
1	Chicken,Pepperoni,Sausage	1.75
2	Chicken,Extra Cheese,Sausage	1.65
3	Extra Cheese,Pepperoni,Saus...	1.60
4	Chicken,Onions,Sausage	1.50
5	Chicken,Extra Cheese,Pepper...	1.45
6	Onions,Pepperoni,Sausage	1.45
7	Extra Cheese,Onions,Sausage	1.35
8	Chicken,Onions,Pepperoni	1.30
9	Chicken,Extra Cheese,Onions	1.20
10	Extra Cheese,Onions,Pepperoni	1.15

EXPERIMENT 1

QUESTION (D)

Amazon wants to identify returning customers who made another purchase shortly after their first purchase.

Write a PostgreSQL query to find all users who made another purchase within 1 to 7 days of an earlier purchase.

Requirements

1. Compare purchases made by the same user only.
2. Ignore comparisons where the same transaction is matched with itself.
3. The second purchase must occur after the first purchase.
4. Same-day purchases must not be considered.
5. The second purchase must occur within 7 days of the first purchase.
6. Display only the unique User IDs that satisfy the above conditions.
7. Sort the output in ascending order of user_id.

SOLUTION:

QUERY:

```
Query Query History
26
27
28
29 CREATE TABLE amazon_purchases (
30     id SERIAL PRIMARY KEY,
31     user_id INT NOT NULL,
32     item_name VARCHAR(255),
33     created_at TIMESTAMP NOT NULL,
34     amount DECIMAL(10, 2)
35 );
36
37 INSERT INTO amazon_purchases (user_id, item_name, created_at, amount) VALUES
38
39 (101, 'Book', '2026-03-01 10:00:00', 15.99),
40 (101, 'Pen', '2026-03-04 14:30:00', 4.50),
41
42 (102, 'Phone', '2026-03-01 09:00:00', 699.99),
43 (102, 'Case', '2026-03-01 18:00:00', 25.00),
44
45 (103, 'Desk', '2026-03-01 11:00:00', 120.00),
46 (103, 'Lamp', '2026-03-10 15:00:00', 35.00),
47
48 (104, 'Shoes', '2026-03-05 08:00:00', 80.00),
49 (104, 'Socks', '2026-03-06 09:00:00', 12.00);
50
51
52
Data Output Messages Notifications
INSERT 0 8
Query returned successfully in 282 msec.
```

EXPERIMENT 1

```
50  
51  
52 SELECT DISTINCT p1.user_id  
53 FROM amazon_purchases p1  
54 JOIN amazon_purchases p2  
55   ON p1.user_id = p2.user_id  
56   AND p1.id != p2.id  
57   AND p2.created_at > p1.created_at  
58   AND p2.created_at::date - p1.created_at::date >= 1  
59   AND p2.created_at::date - p1.created_at::date <= 7  
60 ORDER BY p1.user_id ASC;  
61  
62
```

OUTPUT:

Data Output	Messages	Notifications
Showing rows: 1 to 2 Page No: 1 of 1		
user_id integer		
1	101	
2	104	